Running the Assignment Binaries: the Course Server or Your Own Linux Machine

Introduction

The binaries shipped with this assignment (`mathoracle`, `mindvault`, `casino`) are 64-bit Linux executables (x86-64). They do not run directly on Windows or macOS, and they do not run natively on ARM Linux (Apple Silicon, Raspberry Pi, ...). You do not need to install a virtual machine or any other Linux environment yourself. The CS Department's course server `workbench3.cs.hku.hk` is an x86-64 Ubuntu 24.04 machine with the complete toolchain pre-installed; you reach it with SSH once you are connected to HKUVPN, from your own computer (Option A, recommended). Only if you *already* run your own x86-64 Linux — a native installation, WSL2, or a virtual machine you already have — may you use that instead (Option B).

Quick start (Option A):

```
1. Connect to HKUVPN.
2. ssh <your-cs-account>@workbench3.cs.hku.hk # enter your CS account password
3. Do everything on the server; your own machine is just a terminal.
```

Which option should I choose?

- Windows or macOS machine without Linux: Option A.
- Apple Silicon or any other ARM machine: Option A — the binaries are x86-64 and do not run natively on ARM Linux. Use the course server instead.
- You already run x86-64 Linux (native, WSL2, or an existing VM): either option. Option A keeps your work in one place and needs no setup; Option B works without VPN.
- A locked-down machine that blocks VPN or SSH: contact the course team.

Option A: the CS course server (recommended)

A.1 What you need

- **A CS account.** Enrolled students, TAs and staff all have one; the username is your CS account (of the form `u3001234`). If you do not know its password or forgot it, set a new one at `https://intranet.cs.hku.hk/account`.
- **HKUVPN.** Install the VPN client from ITS (`https://its.hku.hk/services/network-connectivity/hkuvpn/`) and connect it *before* you ssh. The workbench hosts are reachable only through HKUVPN — even when you are on the HKU campus network: being connected to campus Wi-Fi or LAN does not replace the VPN.

A.2 Log in

```
ssh <your-cs-account>@workbench3.cs.hku.hk
# for example: ssh u3001234@workbench3.cs.hku.hk
# then enter your CS account password
```

The first time you connect, ssh shows the server's host key fingerprint and asks whether to trust it; answer `yes`. You then land in your home directory on an x86-64 Ubuntu 24.04 machine: editing, compiling, running and debugging all happen there, while your own computer only displays the terminal.

Use `tmux` (pre-installed) to keep your session running if the connection drops:

```
tmux new -s comp3355 # start a named session
# ... work inside it ...
# Ctrl-b then d # detach: programs keep running on the server
tmux attach -t comp3355 # re-attach from any later ssh login
```

A.3 Optional: edit with VS Code Remote-SSH

If you prefer a full editor over a terminal, install the *Remote - SSH* extension in VS Code and add the following to your SSH configuration file (`~/.ssh/config` on macOS/Linux; on Windows it is the file `config` inside the `.ssh` folder of your user profile, e.g. `C:\Users\<your-username>\.ssh\config`):

```
Host hku-cs
    HostName workbench3.cs.hku.hk
    User <your-cs-account>
```

Then run "Remote-SSH: Connect to Host..." from the command palette, pick `hku-cs`, enter your CS password, and open your home directory as the working folder. Editing, the integrated terminal and gdb all run on the server; the VPN must be connected as usual.

A.4 Toolchain: already installed

`gcc`, `gdb`, `make`, `file`, `git`, `tmux`, `python3` and `pwntools` are pre-installed on the server; there is nothing to apt or pip install. Check pwntools with:

```
python3 -c "from pwn import *; print('pwntools OK')"
```

Optional but recommended: `pwndbg`, the gdb plugin used throughout the README (https://github.com/pwndbg/pwndbg). The server gives you no sudo, so fetch the pre-built portable release into your home directory instead of building it:

```
cd ~
release_url=https://github.com/pwndbg/pwndbg/releases/download
curl -LO \
  "$release_url/2026.09.15/pwndbg_2026.09.15_x86_64-portable.tar.xz"
tar xf pwndbg_2026.09.15_x86_64-portable.tar.xz
```

Unpacking creates `~/pwndbg`. Its wrapper, `~/pwndbg/bin/pwndbg`, runs gdb with the plugin already loaded. To use it whenever you type `gdb` in Bash, run these commands once:

```
echo 'alias gdb="$HOME/pwndbg/bin/pwndbg"' >> ~/.bashrc
source ~/.bashrc
```

The alias also applies to future terminal sessions. Use `/usr/bin/gdb` if you want plain GDB. After copying the assignment, try the debugging example in section A.5.

Plain `gdb` still works exactly as described in the README; every `pwndbg` command the README uses (`telescope`, `vis-heap-chunks`, ...) comes from the wrapper.

The server runs with ASLR enabled (`cat /proc/sys/kernel/randomize_va_space` prints `2`), the same setting as the course machines, so the debugging advice in the README applies unchanged.

A.5 Get the assignment onto the server

From a terminal on your own machine (VPN connected), copy the assignment folder into your server home directory:

```
scp -r assignment <your-cs-account>@workbench3.cs.hku.hk:~/
```

Equally fine: drag the folder into the VS Code Remote-SSH file explorer, or upload and unzip an archive. Then, on the server, restore the executable bit, which transfers often lose, and run the sanity check:

```
chmod +x ~/assignment/task1_secureAI_mathoracle/mathoracle
chmod +x ~/assignment/task2_mindvault_memory_clinic/mindvault
chmod +x ~/assignment/task3_secure_casino/casino
cd ~/assignment/task1_secureAI_mathoracle
file ./mathoracle
./mathoracle
cat /proc/sys/kernel/randomize_va_space
```

`file` must report "ELF 64-bit LSB pie executable, x86-64", the binary must print its banner (quit with Ctrl-C), and the last command must print `2`.

From the same task directory, start the debugger with:

```
gdb ./mathoracle
```

Run each binary and exploit script from its own task directory so the program can find `flag.txt`.

A.6 Sharing the machine

- `workbench3` is shared by the whole course: run only what your assignment needs, and clean up processes you no longer use (`tmux kill-session -t comp3355`, or kill your own stray binaries).
- Keep your working copy in your home directory on the server.
- The assignment rule still applies: run the binaries we shipped, do not recompile them from the provided source code.

Option B: your own x86-64 Linux environment

If you *already* run Linux on x86-64 — `uname -m` must print `x86_64` — you may do the whole assignment there: a native installation, WSL2 on Windows, or a virtual machine you already have set up. ARM Linux (for example a Linux VM on an Apple Silicon Mac) cannot run the binaries natively; use Option A instead. This tutorial no longer walks you through installing WSL2, a VM or Docker: if you do not have an x86-64 Linux environment yet, Option A is the supported route. For local work, we recommend Ubuntu 24.04 on x86-64.

B.1 Install the toolchain

In a terminal run:

```
sudo apt update
sudo apt install -y gcc gdb python3 python3-pip tmux git file
```

Then install pwntools, the Python library used by the assignment scripts. Ubuntu 24.04 and newer block pip installs into the system Python (PEP 668), so use one of these two:

```
sudo apt install -y python3-pwntools
# or:
python3 -m pip install --user --break-system-packages pwntools
```

On an older release such as 22.04 the plain form works:

```
python3 -m pip install --user pwntools
```

Optional but recommended: `pwndbg`, a gdb plugin that makes stack and heap inspection much more readable:

```
git clone https://github.com/pwndbg/pwndbg
cd pwndbg && ./setup.sh
```

Check that the library imports:

```
python3 -c "from pwn import *; print('pwntools OK')"
```

B.2 Copy the assignment and sanity-check

Keep your working copy on the Linux filesystem (your home directory), not on `/mnt/c` or a shared folder: access is faster on the Linux filesystem, which also preserves executable permissions. After copying, restore the executable bit and run the same sanity check as at the end of section A.5:

```
chmod +x ~/assignment/task1_secureAI_mathoracle/mathoracle
chmod +x ~/assignment/task2_mindvault_memory_clinic/mindvault
chmod +x ~/assignment/task3_secure_casino/casino
cd ~/assignment/task1_secureAI_mathoracle
file ./mathoracle
./mathoracle
cat /proc/sys/kernel/randomize_va_space
```

Troubleshooting

- `ssh: connect to host workbench3.cs.hku.hk port 22: Connection timed out` — HKUVPN is not connected (or dropped); being on the campus network is not enough. Connect the VPN and retry.
- `Permission denied, please try again.` at the password prompt — wrong CS account password. Set a new one at `https://intranet.cs.hku.hk/account` and retry.
- VS Code Remote-SSH hangs on "Setting up SSH connection" — usually the VPN; also note the password prompt appears in the palette at the top of the VS Code window, not in a terminal.
- `bash: ./mathoracle: Permission denied` — the executable bit was lost during transfer: run `chmod +x mathoracle`.
- `bash: ./mathoracle: cannot execute binary file: Exec format error` — wrong architecture: `uname -m` must print `x86_64`; on an ARM machine use Option A.
- `from pwn import *` raises `ModuleNotFoundError` — on your own machine, re-run the install with `python3 -m pip` exactly as shown in B.1; on the server pwntools is pre-installed for `/usr/bin/python3`.
- Work lost after a network drop — run inside `tmux` (section A.2).
- Remember the assignment rule: run the binaries we shipped, do not recompile them from the provided source code.